ASSESSMENT-1

CSA1707- ARTIFICAL INTELLIGENCE

Q1. Water Jug Problem

Problem Understanding

Given two jugs:

- One **4-gallon jug**
- One **3-gallon jug**

There are no measuring marks on either jug. The available operations are:

- Fill a jug completely from the pump.
- Empty a jug onto the ground.
- Pour water from one jug to another until one becomes empty or the other becomes full.

Objective: Obtain exactly **2 gallons** of water in the **4-gallon jug**.

Method Selected

The problem is solved using **State Space Search**, an **Uninformed Search** technique.

Each state is represented as:

(Amount in 4-Gallon Jug, Amount in 3-Gallon Jug)

Example:

- (0,0) → Both jugs are empty.
- (4,2) → 4-gallon jug is full and 3-gallon jug contains 2 gallons.

Solution Process

Step	Action	State (4G,3G)
1	Initially both jugs are empty	(0,0)
2	Fill the 3-gallon jug	(0,3)
3	Pour the 3-gallon jug into the 4-gallon jug	(3,0)
4	Fill the 3-gallon jug again	(3,3)
5	Pour water into the 4-gallon jug until it becomes full	(4,2)
6	Empty the 4-gallon jug	(0,2)

Step	Action	State (4G,3G)
7	Pour the remaining 2 gallons into the 4-gallon jug	(2,0)

Python Program

```
# Water Jug Problem (4 Gallon and 3 Gallon)

state = (0, 0)
steps = []

steps.append(("Initial", state))

state = (0, 3)
steps.append(("Fill 3-gallon jug", state))

state = (3, 0)
steps.append(("Pour 3G -> 4G", state))

state = (3, 3)
steps.append(("Fill 3-gallon jug again", state))

state = (4, 2)
steps.append(("Pour 3G -> 4G until 4G is full", state))

state = (0, 2)
steps.append(("Empty 4-gallon jug", state))

state = (2, 0)
steps.append(("Pour remaining water into 4-gallon jug", state))

for action, s in steps:
```

```
    print(action, ":", s)


print("\nGoal Achieved!")
print("4-Gallon Jug =", state[0], "gallons")
```

Output

```
Initial : (0, 0)
Fill 3-gallon jug : (0, 3)
Pour 3G -> 4G : (3, 0)
Fill 3-gallon jug again : (3, 3)
Pour 3G -> 4G until 4G is full : (4, 2)
Empty 4-gallon jug : (0, 2)
Pour remaining water into 4-gallon jug : (2, 0)


Goal Achieved!
4-Gallon Jug = 2 gallons
```

Result / Interpretation

The Water Jug problem was successfully solved using the **State Space Search** method. The goal state **(2,0)** was reached, which means the **4-gallon jug contains exactly 2 gallons**. This demonstrates how AI uses state-space representation and search strategies to solve problem-solving tasks efficiently.

Q2. Mars Rover Intelligent Agent

Problem Understanding

A Mars Rover is an **Intelligent Agent** that autonomously explores the surface of Mars. It collects scientific data, analyzes rocks and soil, avoids obstacles, and communicates the collected information back to Earth. Since Mars is an uncertain and dynamic environment, the rover must make intelligent decisions without continuous human control.

Method Selected

The most suitable AI architecture is a **Learning Agent with a Model-Based Agent**.

Justification:

- Maintains an internal model of the environment.
- Learns from previous experiences.

- Makes decisions based on current sensor data.
- Adapts to unknown terrain and unexpected obstacles.
- Improves navigation and exploration over time.

Solution

i) Percepts (Inputs received by the rover)

The Mars Rover receives information through various sensors:

- Camera images
- Rock and soil composition
- Temperature
- Atmospheric pressure
- Terrain and obstacles
- Battery level
- Wheel position
- Radiation level
- Current location

---

ii) Operating Environment

Property	**Description**
Observability	Partially Observable
Number of Agents	Single Agent
Nature	Stochastic
Decision Process	Sequential
Environment	Dynamic
State Space	Continuous
Knowledge	Partially Unknown

---

iii) Actions Performed

The Mars Rover can perform the following actions:

- Move Forward
- Turn Left
- Turn Right
- Capture Images
- Collect Rock Samples
- Analyze Soil
- Avoid Obstacles
- Send Data to Earth
- Recharge using Solar Panels
- Stop during hazardous conditions

---

iv) Performance Measure

The rover's performance is evaluated based on:

- Accurate sample collection
- Safe navigation
- Obstacle avoidance
- Scientific data quality
- Minimum energy consumption
- Successful communication with Earth
- Completion of exploration mission
- Reduced mission time

---

v) Suitable Agent Architecture

Learning Agent with Model-Based Architecture

Reason:

- Learns from experience.
- Stores previous observations.
- Predicts unseen conditions.
- Makes intelligent decisions in uncertain environments.

- Continuously improves its performance.

Python Program (Simple Demonstration)

```
print("====== Mars Rover Intelligent Agent ======")

percepts = [
    "Camera Images",
    "Rock Samples",
    "Temperature",
    "Battery Level",
    "Terrain Information"
]

actions = [
    "Move Forward",
    "Turn Left",
    "Turn Right",
    "Capture Image",
    "Collect Sample",
    "Analyze Soil",
    "Transmit Data"
]

print("\nPercepts:")
for p in percepts:
    print("-", p)

print("\nActions:")
for a in actions:
    print("-", a)
```

```
print("\nPerformance Measure:")
print("- Safe Navigation")
print("- Accurate Sample Collection")
print("- Efficient Energy Usage")
print("- Successful Data Transmission")

print("\nSuitable Agent: Learning Agent with Model-Based Architecture")
```

Output

```
====== Mars Rover Intelligent Agent ======

Percepts:
- Camera Images
- Rock Samples
- Temperature
- Battery Level
- Terrain Information

Actions:
- Move Forward
- Turn Left
- Turn Right
- Capture Image
- Collect Sample
- Analyze Soil
- Transmit Data

Performance Measure:
- Safe Navigation
- Accurate Sample Collection
- Efficient Energy Usage
```

- Successful Data Transmission

Suitable Agent: Learning Agent with Model-Based Architecture

Result / Interpretation

The Mars Rover is best represented as a **Learning Agent with a Model-Based Architecture** because it can perceive its environment, maintain an internal model, learn from experience, and make intelligent decisions in the uncertain conditions of Mars. This enables autonomous exploration, efficient navigation, and successful scientific missions.

Q3. 8-Queens Problem

Problem Understanding

The **8-Queens Problem** is a classic Artificial Intelligence search problem. The objective is to place **8 queens** on an **8 × 8 chessboard** such that no two queens attack each other.

The following conditions must be satisfied:

- No two queens should be in the same **row**.
- No two queens should be in the same **column**.
- No two queens should be on the same **diagonal**.

The goal is to find a valid arrangement that satisfies all these constraints.

Method Selected

The problem is solved using **State Space Search** with the **Backtracking Algorithm**.

Justification

- Queens are placed one column at a time.
- Before placing a queen, the algorithm checks whether the position is safe.
- If no safe position exists, it backtracks to the previous column and tries another position.
- This avoids exploring unnecessary states and efficiently finds a valid solution.

Problem Formulation

Initial State

An empty 8 × 8 chessboard with no queens placed.

State Space

Each state represents a partial arrangement of queens on the chessboard.

Example:

Q . . . . . .

. . Q . . . . .

. . . . Q . . .

Operators

- Place a queen in a safe row of the current column.
- Move to the next column.
- Backtrack if no safe position exists.

Goal State

A chessboard containing **8 queens**, where no queen attacks another queen.

Path Cost

Each successful queen placement has a cost of **1**.

Total cost to reach the solution = **8**.

Solution Process

1. Start with an empty chessboard.
2. Place the first queen in the first column.
3. Move to the next column and place a queen in a safe position.
4. Continue placing queens column by column.
5. If a conflict occurs, remove the previous queen and try another safe position.
6. Repeat until all eight queens are placed safely.

Python Program

```
N = 8

board = [-1] * N

def isSafe(row, col):
    for i in range(col):
        if board[i] == row or abs(board[i] - row) == abs(i - col):
            return False
    return True

def solve(col):
    if col == N:
        return True

    for row in range(N):
        if isSafe(row, col):
            board[col] = row

            if solve(col + 1):
                return True

    return False

solve(0)

print("Solution Board:\n")

for i in range(N):
    for j in range(N):
        if board[j] == i:
```

```
            print("Q", end=" ")
        else:
            print(".", end=" ")
    print()
```

Output

Solution Board:

```
Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .
```

Result / Interpretation

The **8-Queens Problem** was successfully solved using the **Backtracking Search Algorithm**. The algorithm systematically explores possible queen placements and backtracks whenever a conflict occurs. A valid solution is obtained where no two queens attack each other, demonstrating the effectiveness of state-space search and backtracking in solving constraint satisfaction problems.

Q4. OLA Cab Booking Problem

Problem Understanding

A customer wants to travel from one location to another using the **OLA Cab Booking** mobile application. The customer enters the pickup and destination locations, views the available cab types (Micro, Mini, Sedan, Prime, Shared, etc.), selects a preferred cab, and confirms the booking. The system must identify the most suitable cab and complete the booking process.

Method Selected

Type of Problem-Solving Agent: Goal-Based Agent

Justification

A **Goal-Based Agent** is suitable because its primary objective is to help the customer successfully reach the destination. The agent considers the customer's input (pickup, destination, and preferred cab type), checks cab availability, calculates the fare, and books the cab to achieve the user's goal.

Solution Process

1. Customer enters the pickup location.
2. Customer enters the destination.
3. System displays available cab types.
4. Customer selects a preferred cab.
5. System checks whether the selected cab is available.
6. If available, the fare is calculated.
7. Booking is confirmed and driver details are displayed.
8. If not available, the customer is informed to choose another cab.

Pseudocode

START

Input Pickup Location

Input Destination

Display Available Cab Types

User selects Cab Type

IF Cab is Available THEN

    Calculate Fare

    Confirm Booking

    Assign Driver

    Display Driver Details

```
ELSE
    Display "Cab Not Available"
ENDIF

END
```

Python Program

```
pickup = input("Enter Pickup Location: ")
destination = input("Enter Destination: ")

cab_types = ["Micro", "Mini", "Sedan", "Prime", "Shared"]

print("\nAvailable Cab Types:")
for i in range(len(cab_types)):
   print(i + 1, ".", cab_types[i])

choice = int(input("\nSelect Cab (1-5): "))

fare = {
   "Micro": 120,
   "Mini": 160,
   "Sedan": 220,
   "Prime": 300,
   "Shared": 90
}

selected = cab_types[choice - 1]

print("\n----- Booking Confirmation -----")
print("Pickup      :", pickup)
print("Destination :", destination)
```

```
print("Cab Type    :", selected)
print("Estimated Fare : ₹", fare[selected])
print("Driver Assigned Successfully")
print("Booking Confirmed!")
```

Output

```
Enter Pickup Location: Tambaram
Enter Destination: Chennai Airport

Available Cab Types:
1. Micro
2. Mini
3. Sedan
4. Prime
5. Shared

Select Cab (1-5): 2

----- Booking Confirmation -----
Pickup      : Tambaram
Destination : Chennai Airport
Cab Type    : Mini
Estimated Fare : ₹ 160
Driver Assigned Successfully
Booking Confirmed!
```

Result / Interpretation

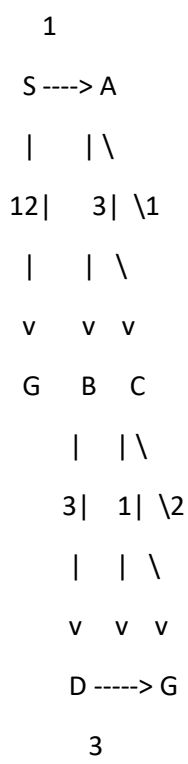
The OLA Cab Booking problem was successfully modeled using a **Goal-Based Agent**. The agent accepts the user's travel details, provides suitable cab options, calculates the fare, and confirms the booking. This demonstrates how goal-based agents make decisions to achieve a specific objective, ensuring an efficient and user-friendly cab booking experience.

Q5. Uniform Cost Search (UCS) – Least Cost Path

Problem Understanding

A logistics company operates a delivery network where packages are transported between warehouses. Each route has an associated travel cost (fuel + time). The objective is to determine the **least-cost path** from the **starting warehouse (S)** to the **destination warehouse (G)** using the **Uniform Cost Search (UCS)** algorithm.

The given weighted graph is:



Method Selected

Uniform Cost Search (UCS)

Justification

Uniform Cost Search is an **Uninformed Search Algorithm** that always expands the node with the **lowest cumulative path cost** first. It guarantees the optimal solution when all edge costs are non-negative.

Solution Process

Step 1

Start from node **S**.

Frontier	Cost
S	0

Step 2

Expand **S**

Possible nodes:

Node	Cost
A	1
G	12

Choose **A** because it has the lowest cost.

Step 3

Expand **A**

New nodes:

Node	Cost
B	4
C	2

Frontier:

Node	Cost
C	2
B	4
G	12

Choose **C**.

Step 4

Expand **C**

New nodes:

Node	Cost
D	3
G	4

Frontier becomes:

Node	Cost
D	3
B	4
G	4

Choose **D**.

Step 5

Expand **D**

New path to G:

Node	Cost
G	6

Existing cost of G is **4**, so UCS keeps the smaller cost.

Step 6

Next minimum-cost node is **G (Cost = 4)**.

Goal reached.

Optimal Path

S

↓

A

↓

C

↓

G

Total Cost

S → A = 1

A → C = 1

C → G = 2

Total Cost = 1 + 1 + 2 = 4

Python Program

```
import heapq

graph = {
    'S': [('A', 1), ('G', 12)],
    'A': [('B', 3), ('C', 1)],
    'B': [('D', 3)],
    'C': [('D', 1), ('G', 2)],
    'D': [('G', 3)],
    'G': []
}

def uniform_cost_search(start, goal):
    priority_queue = [(0, start, [])]
    visited = set()

    while priority_queue:
        cost, node, path = heapq.heappop(priority_queue)

        if node in visited:
```

```
        continue

    visited.add(node)
    path = path + [node]

    if node == goal:
        return cost, path

    for neighbor, weight in graph[node]:
        if neighbor not in visited:
            heapq.heappush(priority_queue,
                           (cost + weight, neighbor, path))

cost, path = uniform_cost_search('S', 'G')

print("Optimal Path :", " -> ".join(path))
print("Minimum Cost :", cost)
```

Output

```
Optimal Path : S -> A -> C -> G
Minimum Cost : 4
```

Result / Interpretation

The **Uniform Cost Search (UCS)** algorithm successfully found the **least-cost path** from the starting warehouse **S** to the destination warehouse **G**. Since UCS always expands the node with the lowest cumulative cost, it guarantees the optimal solution. The optimal delivery route is:

S → A → C → G

with a **minimum total cost of 4**, making it the most efficient route for package transportation.